

Introduction to Software Testing

Chapter 3

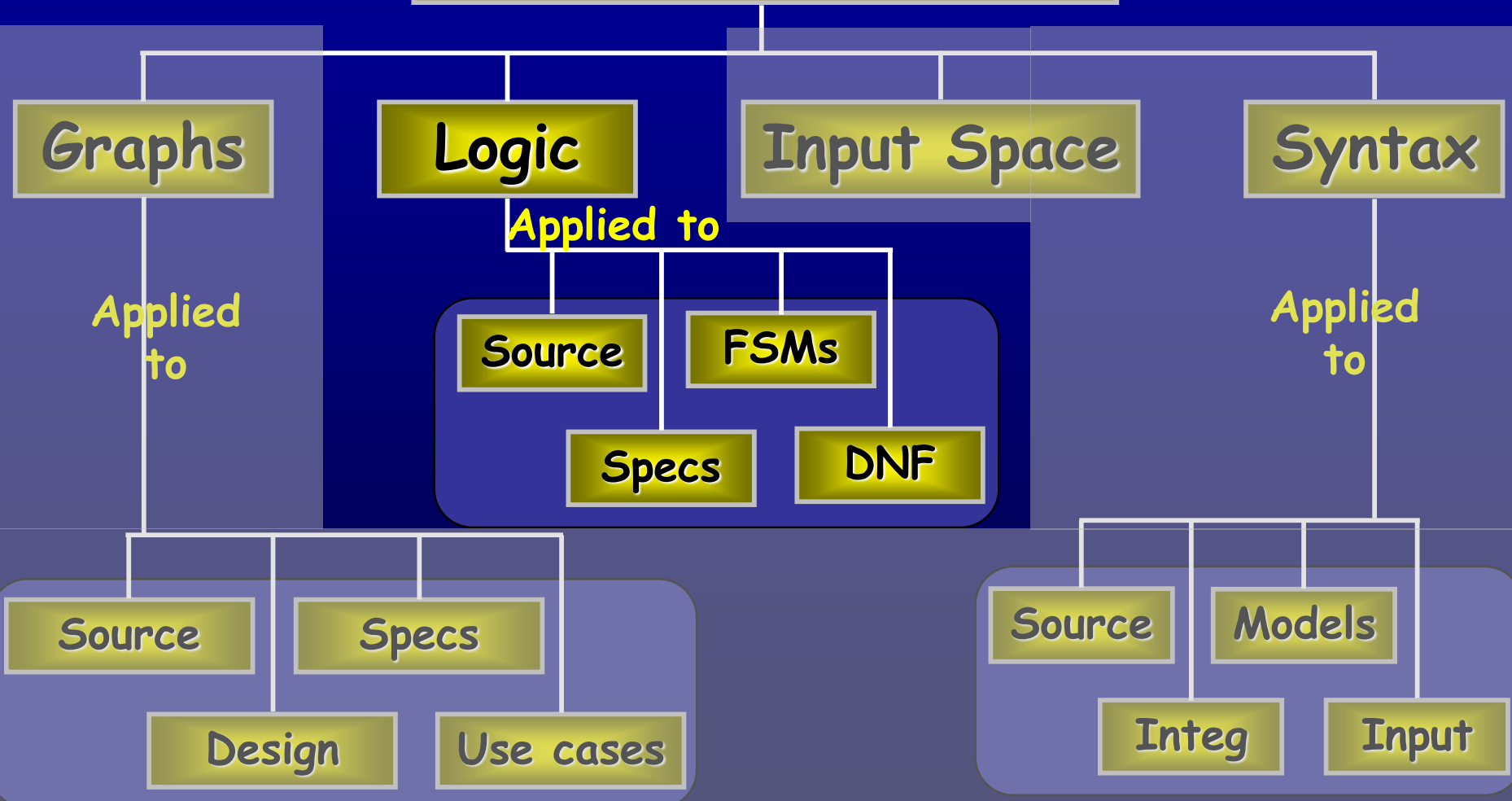
Logic Coverage

Paul Ammann & Jeff Offutt

<http://www.cs.gmu.edu/~offutt/softwaretest/>

Ch. 3 : Logic Coverage

Four Structures for Modeling Software



Framework of logics

- *Propositional logics, Boolean logics*

Focus of the
Chapter

- $p, \wedge, \vee, \neg, \rightarrow, \dots$

- ex: $\text{happy} \wedge (\text{rain} \rightarrow \text{smoke})$

- *1st-order logics*

- $x, p(x), \wedge, \vee, \neg, \forall, \exists, \rightarrow, \dots$

- ex: $(\text{human}(\text{Socrates}) \wedge \forall x(\text{human}(x) \rightarrow \text{mortal}(x)))$
 $\rightarrow \text{mortal}(\text{Socrates})$

- *Higher-order logics, 2nd-order logics*

- $x, p(x), \wedge, \vee, \neg, \forall, \exists, F, \rightarrow, \dots$

- Ex: $\exists F(F(\text{Socrates}) \wedge \forall x \exists y((F(x) \wedge \text{friend}(x,y)) \rightarrow F(y)))$

Covering Logic Expressions (3.1)

- **Logic expressions show up in many situations**
- **Covering logic expressions is required by the US Federal Aviation Administration for safety critical software**
- **Logical expressions can come from many sources**
 - **Decisions in programs**
 - **FSMs and statecharts**
 - **Requirements**
- **Tests are intended to choose some subset of the total number of truth assignments to the expressions**

Logic Predicates and Clauses

- A **predicate** is an expression that evaluates to a **boolean** value
- Predicates can contain
 - **boolean variables**
 - non-boolean variables with $>$, $<$, $==$, $>=$, $<=$, $!=$
 - boolean **function** calls
- Internal structure is created by logical operators
 - $\neg$ – the *negation* operator
 - $\wedge$ – the *and* operator
 - $\vee$ – the *or* operator
 - $\rightarrow$ – the *implication* operator
 - $\oplus$ – the *exclusive or* operator
 - $\leftrightarrow$ – the *equivalence* operator
- A **clause** is a predicate with no logical operators

a non-standard term, should be called ATOM.

Semantics

- Truth table, meaning of propositional logics

	$a < b$	D	$m \geq n * o$	$((a < b) \vee D) \wedge (m \geq n * o)$
1	T	T	T	T
2	T	T	F	F
3	T	F	T	T
4	T	F	F	F
5	F	T	T	T
6	F	T	F	F
7	F	F	T	F
8	F	F	F	F

- a truth assignment,
- an interpretation,
- a row

2014/12/05 stopped here.

Examples

- $(a < b) \vee f(z) \wedge D \wedge (m \geq n * o)$
- Four clauses:
 - $(a < b)$ – relational expression
 - $f(z)$ – boolean-valued function
 - D – boolean variable
 - $(m \geq n * o)$ – relational expression
- Most predicates have few clauses
 - It would be nice to quantify that claim!
- Sources of predicates
 - Decisions in programs
 - Guards in finite state machines
 - Decisions in UML activity graphs
 - Requirements, both formal and informal
 - SQL queries: select a from A b from B where $\text{age}(a) = \text{age}(b)$

$$\{(a, b) \mid a \in A \wedge b \in B \wedge \text{age}(a) = \text{age}(b)\},$$

$$\exists a \in A \exists b \in B \text{ age}(a) = \text{age}(b)$$

Translating from English

- “I am interested in SWE 637 and CS 652”

- *course = swe637 OR course = cs652*

Humans have trouble
translating from
English to Logic

- “If you leave before 6:30 AM, take Braddock to 495, if you leave after 7:00 AM, take Prosperity to 50, then 50 to 495”
- *time < 6:30 → path = Braddock ∨ time > 7:00 → path = Prosperity*
- Hmm ... this is incomplete !
- *time < 6:30 → path = Braddock ∨ time ≥ 6:30 → path = Prosperity*

Testing and Covering Predicates (3.2)

- We use predicates in testing as follows :
 - Developing a model of the software as one or more predicates
 - Requiring tests to satisfy some combination of clauses
- Abbreviations:
 - P is the set of predicates
 - p is a single predicate in P
 - C is the set of clauses in P
 - C_p is the set of clauses in predicate p
 - c is a single clause in C

Predicate and Clause Coverage

- The first (and simplest) two criteria require that each predicate and each clause be evaluated to both true and false

Predicate Coverage (PC) : For each p in P , TR contains two requirements: p evaluates to true, and p evaluates to false.

- When predicates come from conditions on edges, this is equivalent to edge coverage
- PC does not evaluate all the clauses, so ...

Clause Coverage (CC) : For each c in C , TR contains two requirements: c evaluates to true, and c evaluates to false.

Predicate Coverage Example

$$((a < b) \vee D) \wedge (m \geq n * o)$$

predicate coverage

Predicate = true

$$\begin{aligned} & a = 5, b = 10, D = \text{true}, m = 1, n = 1, o = 1 \\ & = (5 < 10) \vee \text{true} \wedge (1 \geq 1 * 1) \\ & = \text{true} \vee \text{true} \wedge \text{TRUE} \\ & = \text{true} \end{aligned}$$

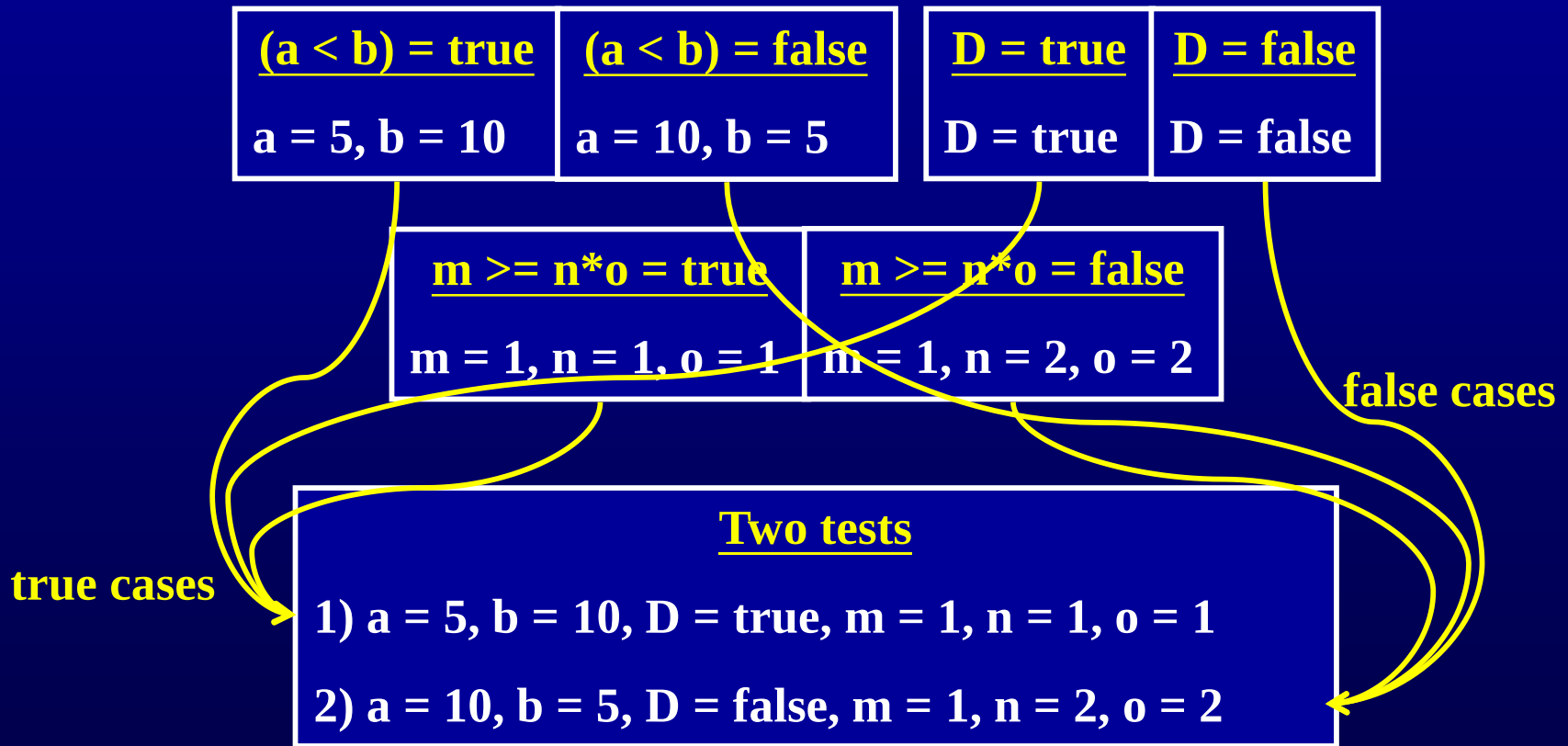
Predicate = false

$$\begin{aligned} & a = 10, b = 5, D = \text{false}, m = 1, n = 1, o = 1 \\ & = (10 < 5) \vee \text{false} \wedge (1 \geq 1 * 1) \\ & = \text{false} \vee \text{false} \wedge \text{TRUE} \\ & = \text{false} \end{aligned}$$

Clause Coverage Example

$$((a < b) \vee D) \wedge (m \geq n * o)$$

Clause coverage



Problems with PC and CC

- **PC does not fully exercise all the clauses, especially in the presence of short circuit evaluation**
- **CC does not always ensure PC**
 - That is, we can satisfy CC without causing the predicate to be both true and false
 - This is definitely not what we want !
- **The simplest solution is to test all combinations ...**

Combinatorial Coverage (CoC)

- CoC requires every possible combination
- Sometimes called Multiple Condition Coverage

For each $\underline{p}$ in $\underline{P}$, TR has test requirements for the clauses in $\underline{C_p}$ to evaluate to each possible combination of truth values.

	$a < b$	D	$m \geq n * o$	$((a < b) \vee D) \wedge (m \geq n * o)$
1	T	T	T	T
2	T	T	F	F
3	T	F	T	T
4	T	F	F	F
5	F	T	T	T
6	F	T	F	F
7	F	F	T	F
8	F	F	F	F

Combinatorial Coverage

- This is simple, neat, clean, and comprehensive ...
- But quite expensive!
- 2^N tests, where N is the number of clauses
 - Impractical for predicates with more than 3 or 4 clauses
- The literature has lots of suggestions – some confusing
- The general idea is simple:

Test each clause independently from the other clauses

- Getting the details right is hard
- What exactly does “independently” mean ?
- The book presents this idea as “*making clauses active*” ...

Active Clauses

- Clause coverage has a weakness : The values do not always make a difference
- Consider the first test for **clause coverage**, which caused each clause to be true:

$$(5 < 10) \vee \text{true} \wedge (1 \geq 1*1)$$

- Only the first clause counts !
- To really test the results of a clause, the clause should be the determining factor in the value of the predicate

Determination :

A clause C_i in predicate p , called the major clause, determines p if and only if the values of the remaining minor clauses C_j are such that changing C_i changes the value of p

- This is considered to **make the clause active**

Determining Predicates (呂宗翰)

$$\underline{p = A \vee B}$$

if **B = true**, p is always true.

so if **B = false**, A determines p .

if **A = false**, B determines p .

$$\underline{p = A \wedge B}$$

if **B = false**, p is always false.

so if **B = true**, A determines p .

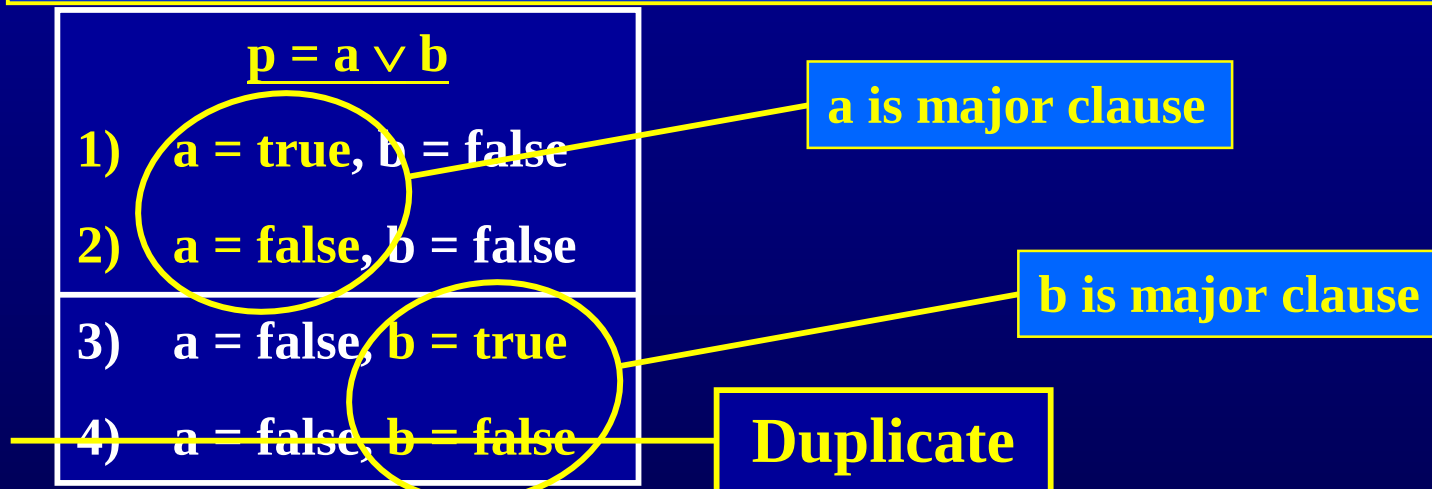
if **A = true**, B determines p .

- **Goal** : Find tests for each clause when the clause determines the value of the predicate
- This is formalized in **several criteria** that have subtle, but very important, differences

Active Clause Coverage (ACC) (吳家賢)

For each p in P and each major clause C_i in C_p , choose minor clauses $C_j, j \neq i$, so that C_i determines p .

TR has two requirements for each C_i : C_i evaluates to true and C_i evaluates to false.



- This is a form of **MCDC (modified condition decision coverage)**, which is required by the FAA for safety critical software
- **Ambiguity** : Do the minor clauses have to have the **same values** when the major clause is true and false?

Resolving the Ambiguity (郭宗霆)

$$\underline{p = a \vee (b \wedge c)}$$

Major clause : **a**

a = true, b = false, c = true

a = false, b = false, **c = false**

Is this allowed ?

- This question caused **confusion** among testers for years
- Considering this carefully leads to **three** separate criteria :
 - Minor clauses **do not** need to be the same
 - Minor clauses **do** need to be the same
 - Minor clauses **force the predicate** to become both true and false

General Active Clause Coverage (GACC)

Motivation: For each p in P and each major clause c_i in C_p , choose minor clauses $c_j, j \neq i$, so that c_i determines p .

TR has two requirements for each c_i : c_i evaluates to true and c_i evaluates to false.

The values chosen for the minor clauses c_j do not need to be the same when c_i is true as when c_i is false, that is,

- $c_j(c_i = \text{true}) = c_j(c_i = \text{false})$ for all c_j OR
- $c_j(c_i = \text{true}) \neq c_j(c_i = \text{false})$ for all c_j .

- This is complicated !
- It is possible to satisfy GACC without satisfying predicate coverage
- We really want to cause predicates to be both true and false !

Restricted Active Clause Coverage (RACC)

Motivation: For each p in P and each major clause c_i in C_p , choose minor clauses $c_j, j \neq i$, so that c_i determines p .

TR has two requirements for each c_i : c_i evaluates to true and c_i evaluates to false.

The values chosen for the minor clauses c_j must be the same when c_i is true as when c_i is false, that is, it is required that $c_j(c_i = \text{true}) = c_j(c_i = \text{false})$ for all c_j .

- This has been a **common interpretation** by aviation developers
- RACC often leads to **infeasible test requirements**
- There is **no logical reason** for such a restriction

Correlated Active Clause Coverage (CACC)

Motivation: For each p in P and each major clause c_i in C_p , choose minor clauses $c_j, j \neq i$, so that c_i determines p .

TR has two requirements for each c_i : c_i evaluates to true and c_i evaluates to false.

The values chosen for the minor clauses c_j must cause p to be true for one value of the major clause c_i and false for the other, that is, it is required that $p(c_i = \text{true}) \neq p(c_i = \text{false})$.

- A more recent interpretation
- **Implicitly** allows minor clauses to have different values
- Explicitly satisfies (**subsumes**) predicate coverage

CACC and RACC

	a	b	c	$a \wedge (b \vee c)$
1	T	T	T	T
2	T	T	F	T
3	T	F	T	T
4	T	F	F	F
5	F	T	T	F
6	F	T	F	F
7	F	F	T	F
8	F	F	F	F

major clause

$P_a : b = \text{true} \text{ or } c = \text{true}$

CACC can be satisfied by choosing any of rows 1, 2, 3 AND any of rows 5, 6, 7 – a total of nine pairs

	a	b	c	$a \wedge (b \vee c)$
1	T	T	T	T
2	T	T	F	T
3	T	F	T	T
4	T	F	F	F
5	F	T	T	F
6	F	T	F	F
7	F	F	T	F
8	F	F	F	F

RACC can only be satisfied by row pairs (1, 5), (2, 6), or (3, 7)

Only three pairs

Inactive Clause Coverage (ICC)

- The active clause coverage criteria ensure that “major” clauses do affect the predicates
- Inactive clause coverage takes the opposite approach – major clauses do not affect the predicates

For each p in P and each major clause c_i in C_p , choose minor clauses $c_j, j \neq i$, so that c_i does not determine p .

TR has four requirements for each c_i :

- (1) c_i evaluates to true with p true,
- (2) c_i evaluates to false with p true,
- (3) c_i evaluates to true with p false, and
- (4) c_i evaluates to false with p false.

General and Restricted ICC

- Unlike ACC, the notion of correlation is not relevant
 - c_i does not determine p , so cannot correlate with p
- Predicate coverage is always guaranteed

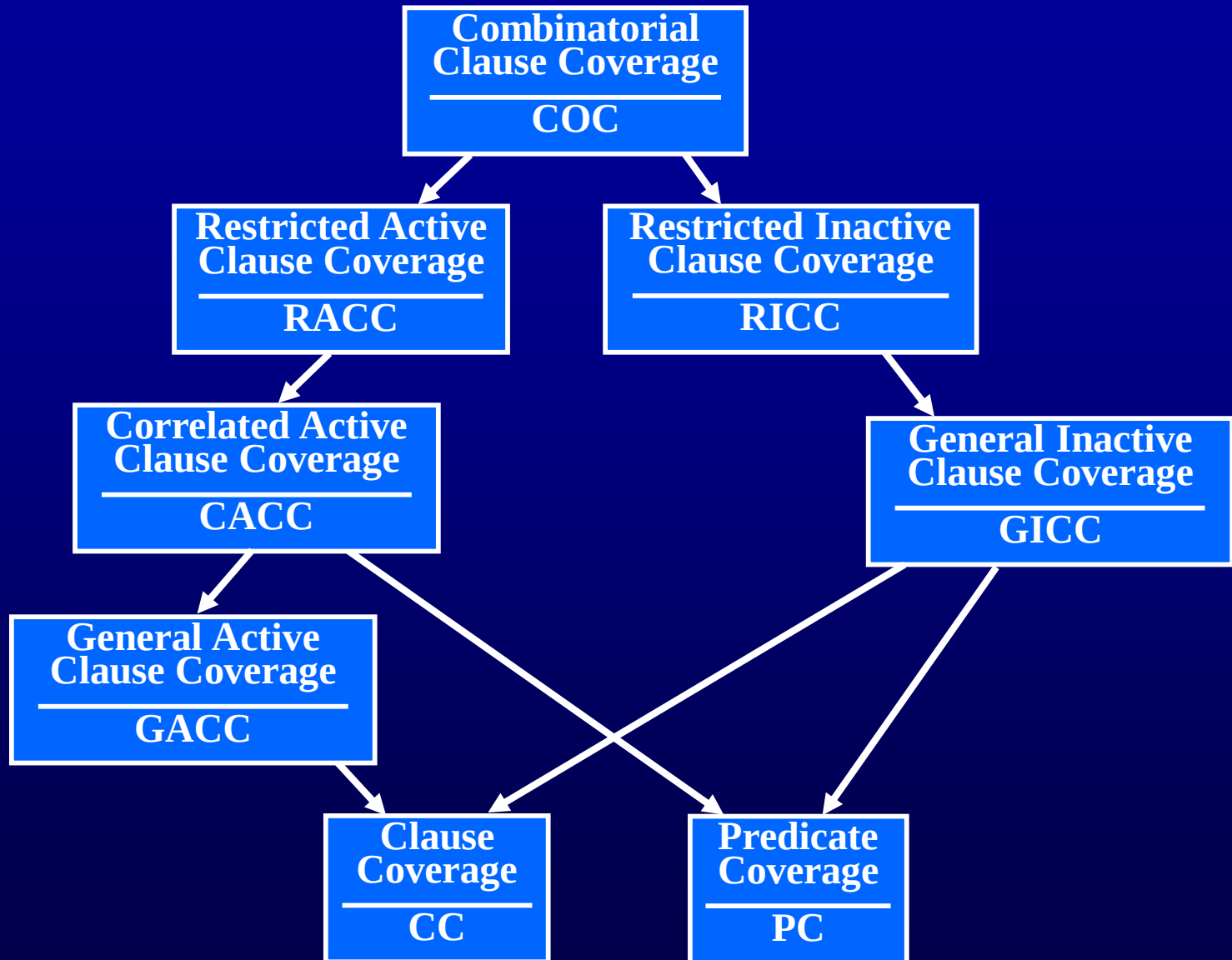
General Inactive Clause Coverage (GICC) : For each p in P and each major clause c_i in C_p , choose minor clauses $c_j, j \neq i$, so that c_i does not determine p .

The values chosen for the minor clauses c_j do not need to be the same when c_i is true as when c_i is false, that is, $c_j(c_i = \text{true}) = c_j(c_i = \text{false})$ for all c_j OR $c_j(c_i = \text{true}) \neq c_j(c_i = \text{false})$ for all c_j .

Restricted Inactive Clause Coverage (RICC) : For each p in P and each major clause c_i in C_p , choose minor clauses $c_j, j \neq i$, so that c_i does not determine p .

The values chosen for the minor clauses c_j must be the same when c_i is true as when c_i is false, that is, it is required that $c_j(c_i = \text{true}) = c_j(c_i = \text{false})$ for all c_j .

Logic Coverage Criteria Subsumption



Making Clauses Determine a Predicate

- Finding values for minor clauses c_j is easy for simple predicates
- But how to find values for more complicated predicates ?
- Definitional approach:
 - $p_{c=true}$ is predicate p with every occurrence of c replaced by *true*
 - $p_{c=false}$ is predicate p with every occurrence of c replaced by *false*

$$\underline{p = ((a < b) \vee D) \wedge (m \geq n * o)}$$

$$\begin{aligned} p_{(a < b)=true} &= (true \vee D) \wedge (m \geq n * o) \\ &= (m \geq n * o) \end{aligned}$$

$$\begin{aligned} p_{(a < b)=false} &= (false \vee D) \wedge (m \geq n * o) \\ &= D \wedge (m \geq n * o) \end{aligned}$$

Making Clauses Determine a Predicate

- Finding values for minor clauses c_j is easy for simple predicates
 - But how to find values for more complicated predicates ?
 - Definitional approach:
 - $p_{c=true}$ is predicate p with every occurrence of c replaced by **true**
 - $p_{c=false}$ is predicate p with every occurrence of c replaced by **false**
 - To find values for the minor clauses, connect $p_{c=true}$ and $p_{c=false}$ with exclusive OR
- $$p_c = p_{c=true} \oplus p_{c=false}$$
- After solving, p_c describes exactly the values needed for c to determine p

Examples

$$\underline{p = a \vee b}$$

$$\begin{aligned} p_a &= p_{a=\text{true}} \oplus p_{a=\text{false}} \\ &= (\text{true} \vee b) \text{ XOR } (\text{false} \vee b) \\ &= \text{true} \text{ XOR } b \\ &= \neg b \end{aligned}$$

$$\underline{p = a \wedge b}$$

$$\begin{aligned} p_a &= p_{a=\text{true}} \oplus p_{a=\text{false}} \\ &= (\text{true} \wedge b) \oplus (\text{false} \wedge b) \\ &= b \oplus \text{false} \\ &= b \end{aligned}$$

$$\underline{p = a \vee (b \wedge c)}$$

$$\begin{aligned} p_a &= p_{a=\text{true}} \oplus p_{a=\text{false}} \\ &= (\text{true} \vee (b \wedge c)) \oplus (\text{false} \vee (b \wedge c)) \\ &= \text{true} \oplus (b \wedge c) \\ &= \neg (b \wedge c) \\ &= \neg b \vee \neg c \end{aligned}$$

- “**NOT $b \vee \text{NOT } c$** ” means either b or c can be false
- RACC requires the same choice for both values of a , CACC does not

Repeated Variables

- The definitions in this chapter yield the same tests no matter how the predicate is expressed
- $(a \vee b) \wedge (c \vee b) == (a \wedge c) \vee b$
- $(a \wedge b) \vee (b \wedge c) \vee (a \wedge c)$
 - Only has 8 possible tests, not 64
- Use the simplest form of the predicate, and ignore contradictory truth table assignments

To simplify test analysis ?

- How do you know this is equivalent ?
- How do you know one b is equivalent to 2 b's ?

2013/12/05 stopped here.

A More Subtle Example

$$\underline{p = (a \wedge b) \vee (a \wedge \neg b)}$$

$$\begin{aligned} p_a &= p_{a=\text{true}} \oplus p_{a=\text{false}} \\ &= ((\text{true} \wedge b) \vee (\text{true} \wedge \neg b)) \oplus ((\text{false} \wedge b) \vee (\text{false} \wedge \neg b)) \\ &= (b \vee \neg b) \oplus \text{false} \\ &= \text{true} \oplus \text{false} \\ &= \text{true} \end{aligned}$$

$$\underline{p = (a \wedge b) \vee (a \wedge \neg b)}$$

$$\begin{aligned} p_b &= p_{b=\text{true}} \oplus p_{b=\text{false}} \\ &= ((a \wedge \text{true}) \vee (a \wedge \neg \text{true})) \oplus ((a \wedge \text{false}) \vee (a \wedge \neg \text{false})) \\ &= (a \vee \text{false}) \oplus (\text{false} \vee a) \\ &= a \oplus a \\ &= \text{false} \end{aligned}$$

- **a** always determines the value of this predicate
- **b** never determines the value – **b** is **irrelevant** !

Infeasible Test Requirements

- Consider the predicate:

$$(a > b \wedge b > c) \vee c > a$$

- $(a > b) = \text{true}, (b > c) = \text{true}, (c > a) = \text{true}$ is infeasible
- As with graph-based criteria, infeasible test requirements have to be recognized and ignored
- Recognizing infeasible test requirements is hard, and in general, undecidable
- Software testing is **inexact** – engineering, not science

Logic Coverage Summary

- Predicates are often **very simple**—in practice, most have less than 3 clauses
 - In fact, most predicates only have one clause !
 - With only clause, PC is enough
 - With 2 or 3 clauses, CoC is practical
 - Advantages of ACC and ICC criteria significant for large predicates
 - CoC is impractical for predicates with many clauses
- **Control software** often has many complicated predicates, with lots of clauses
- **Question** ... why don't complexity metrics count the number of clauses in predicates?

Introduction to Software Testing

Chapter 3.3

Logic Coverage for Source Code

Paul Ammann & Jeff Offutt

<http://www.cs.gmu.edu/~offutt/softwaretest/>

Logic Expressions from Source (俊瑋講)

- Predicates are derived from decision statements in programs
- In programs, most predicates have less than four clauses
 - Wise programmers actively strive to keep predicates simple
- When a predicate only has one clause, COC, ACC, ICC, and CC all collapse to predicate coverage (PC)
- Applying logic criteria to program source is hard because of reachability and controllability:
 - Reachability : Before applying the criteria on a predicate at a particular statement, we have to **get to** that statement
 - Controllability : We have to **find input values** that indirectly assign values to the variables in the predicates
 - Variables in the predicates that are not inputs to the program are called **internal variables**
- These issues are illustrated through an example in the following slides ...

Thermostat (pg 1 of 2) (庭蔡講)

```
1 // Jeff Offutt--October 2010
2 // Programmable Thermostat
3 import java.io.*;
4 class thermostat
5 {
6     private Heater myHeater;
7     // Decide whether to turn the heater on, and for how long.
8     public boolean turnHeaterOn (
9         int curTemp, /* Current temperature reading */
10        int thresholdDiff, /* Temp difference until we turn heater on */
11        Minutes timeSinceLastRun, /* Time since heater stopped */
12        Minutes minLag, /* How long I need to wait */
13        Time timeOfDay, /* current time (Hours and minutes) */
14        Day dayOfWeek, /* Monday, Tuesday, ... */
15        Settings programmedSettings [], /* User's program, by day */
16        boolean Override, /* Has user overridden the program */
17        int overTemp /* OverridingTemp */
18    )
```

Thermostat (pg 2 of 2) (庭蔡講)

```
19 {
20   int desiredTemp;
21   // getPeriod() translates time into Morning, Day, Evening, Night
22   desiredTemp = programmedSettings [dayOfWeek].getDesiredTemp
23       (getPeriod [TimeOfDay]);
24   if (((curTemp < desiredTemp - thresholdDiff) ||
25       (Override && curTemp < overTemp - thresholdDiff)) &&
26       timeSinceLastRun.greaterThan (minLag))
27   { // Turn on the heater
28       // How long? Assume 1 minute per degree (Fahrenheit)
29       int timeNeeded = curTemp - desiredTemp;
30       if (Override)
31           timeNeeded = curTemp - overTemp;
32       myHeater.setRunTime (timeNeeded);
33       return (true);
33   }
34   else
35       return (false);
36 } // End turnHeaterOn
37 } // End class
```

Two Thermostat Predicates (庭蔡講)

24-26: (((curTemp < desiredTemp - thresholdDiff) ||
(Override && curTemp < overTemp - thresholdDiff)) &&
timeSinceLastRun.greaterThan (minLag))

30: (Override)

Simplify

a : curTemp < desiredTemp - thresholdDiff

b : Override

c : curTemp < overTemp - thresholdDiff

d : timeSinceLastRun.greaterThan (minLag)

24-26: (a || (b && c)) && d

30: b

Reachability for Thermostat Predicates

24-26 : True

49: (a || (b && c)) && d

curTemp < desiredTemp - thresholdDiff

Need to solve for the internal variable desiredTemp

programmedSettings [dayOfWeek].getDesiredTemp (getPeriod [TimeOfDay])

programmedSettings [Monday].setDesiredTemp (Morning, 69)

Predicate Coverage (*true*)

(a || (b && c)) && d

a : true

b : true

c : true

d : true

curTemp < desiredTemp – thresholdDiff : true

Override : true

curTemp < overTemp – thresholdDiff : true

timeSinceLastRun.greaterThan (minLag) : true

programmedSettings [Monday].setDesiredTemp (Morning, 69)

// dayOfWeek = Monday

// timeOfDay = 8:00

curTemp = 63

63 < 69 – 5

Override : true

overTemp = 70

63 < 70 – 5

timeSinceLastRun.setValue (12)

minLag = 10

Predicate Coverage (*false*)

$(a \parallel (b \ \&\& \ c)) \ \&\& \ d$

a : false

b : false

c : false

d : false

curTemp < desiredTemp – thresholdDiff : **false**

Override : **false**

curTemp < overTemp – thresholdDiff : **false**

timeSinceLastRun.greaterThan (minLag) : **false**

programmedSettings [**Monday**].setDesiredTemp (**Morning, 69**)

// dayOfWeek = **Monday**

// timeOfDay = **8:00**

curTemp = **66**

66 < 69 – 5

Override : **false**

overTemp = **70**

66 < 70 – 5

timeSinceLastRun.setValue (**8**)

minLag = **10**

Correlated Active Clause Coverage (1 of 5)

$$P_a = ((a \parallel (b \&\& c)) \&\& d) \oplus ((a \parallel (b \&\& c)) \&\& d)$$

$$((T \parallel (b \&\& c)) \&\& d) \oplus ((F \parallel (b \&\& c)) \&\& d)$$

$$(T \&\& d) \oplus ((b \&\& c) \&\& d)$$

$$d \oplus ((b \&\& c) \&\& d)$$

$$!(b \&\& c) \&\& d$$

$$(!b \parallel !c) \&\& d$$

Check with the logic coverage web app

<http://cs.gmu.edu:8080/offutt/coverage/LogicCoverage>

Correlated Active Clause Coverage (2 of 5)

(a (b && c)) && d				
a:	T	t	f	t (row 3)
	F	t	f	t (row 13)
b:	f	T	t	t (row 9)
	f	F	t	t (row 13)
c:	f	t	T	t (row 9)
	f	t	F	f (row 11)
d:	t	t	t	T (row 1)
	t	t	t	F (row 2)

duplicates

Six tests needed for CACC on Thermostat

Correlated Active Clause Coverage (3 of 5)

	curTemp	desiredTemp	thresholdDiff
curTemp < desiredTemp - thresholdDiff	63	69	5
!(curTemp < desiredTemp - thresholdDiff)	66	69	5

desiredTemp = programmedSettings [Monday].setDesiredTemp (Morning, 69)
dayOfWeek = Monday
timeOfDay = 8:00

	Override
Override	T
!Override	F

These values then need
to be placed into calls to
turnHeaterOn() to satisfy
the 6 tests for CACC

	curTemp	overTemp	thresholdDiff
curTemp < overTemp - thresholdDiff	63	70	5
!(curTemp < overTemp - thresholdDiff)	66	70	5

	timeSinceLastRun	minLag
timeSinceLastRun.greaterThan (minLag)	12	10
!(timeSinceLastRun.greaterThan (minLag))	8	10

Correlated Active Clause Coverage (4 of 5)

desiredTemp = programmedSettings [Monday].setDesiredTemp (Morning, 69)

1. T t f t

a = T : curTemp = 63; c = f : curTemp = 66

turnHeaterOn (63/66, 5, 12, 10, 8:00, Monday, programmedSettings, true, 70)

2. F t f t

turnHeaterOn (66, 5, 12, 10, 8:00, Monday, programmedSettings, true, 70)

3. f T t t

a = f : curTemp = 66; c = t : curTemp = 63

turnHeaterOn (63/66, 5, 12, 10, 8:00, Monday, programmedSettings, true, 70)

4. f t F f

turnHeaterOn (66, 5, 8, 10, 8:00, Monday, programmedSettings, true, 70)

5. t t t T

turnHeaterOn (63, 5, 12, 10, 8:00, Monday, programmedSettings, true, 70)

6. t t t F

turnHeaterOn (63, 5, 8, 10, 8:00, Monday, programmedSettings, true, 70)

Correlated Active Clause Coverage (5 of 5)

- Tests **1** and **3** are infeasible with the values we chose
- But we can choose different values for clause **c**
- **curTemp** is fixed by the solution to clause **a**
- **thresholdDiff** is also fixed by the solution to clause **a**
- So we choose different values for **overtemp** ...

1. T t f t

turnHeaterOn (63, 5, 12, 10, 8:00, Monday, programmedSettings, true, 62)

3. f T t t

turnHeaterOn (66, 5, 12, 10, 8:00, Monday, programmedSettings, true, 66)

Program Transformation Issues

```
if ((a && b) || c) {  
    S1;  
}  
else {  
    S2;  
}
```


Transform (1)?

```
if (a) {  
    if (b)  
        S1;  
    else {  
        if (c)  
            S1;  
        else  
            S2;  
    }  
}  
else {  
    if (c)  
        S1;  
    else  
        S2;  
}
```


Transform (2)?

```
d = a && b;  
e = d || c;  
if (e) {  
    S1;  
}  
else {  
    S2;  
}
```

Problems with Transformed Programs

- **Maintenance is certainly harder with Transform (1)**
 - Not recommended!
- **Coverage on Transform (1)**
 - PC on transform does not imply CACC on original
 - CACC on original does not imply PC on transform
- **Coverage on Transform (2)**
 - Structure used by logic criteria is “lost”
 - Hence CACC on transform 2 only requires 3 tests
 - Note: Mutation analysis (Chapter 5) addresses this problem
- **Bottom Line: Logic coverage criteria are there to help you!**

a	b	c	$(a \wedge b) \vee c$	CACC	PC	CACC (2)
T	T	T	T		X	
T	T	F	T	X		X
T	F	T	T	X	X	X
T	F	F	F	X	X	
F	T	T	T		X	
F	T	F	F	X		X
F	F	T	T			
F	F	F	F		X	

Summary : Logic Coverage for Source Code

- **Predicates** appear in decision statements
 - if, while, for, etc.
- Most predicates have less than **four clauses**
 - But some applications have predicates with many clauses
- The hard part of applying logic criteria to source is resolving the **internal variables**
- Sometimes setting variables requires calling **other methods**
- **Non-local variables** (class, global, etc.) are also input variables if they are used
- If an input variable is changed within a method, it is treated as an **internal variable** thereafter
- To maximize effect of logic coverage criteria:
 - Avoid transformations that hide predicate structure

Introduction to Software Testing

Chapter 3.4

Logic Coverage for Specifications

Paul Ammann & Jeff Offutt

<http://www.cs.gmu.edu/~offutt/softwaretest/>

Specifications in Software

- Specifications can be **formal** or **informal**
 - Formal specs are usually expressed **mathematically**
 - Informal specs are usually expressed in ***natural language***
- Lots of **formal languages** and **informal styles** are available
- Most specification languages include **explicit logical expressions**, so it is very easy to apply logic coverage criteria
- Implicit logical expressions in natural-language specifications should be **re-written** as explicit logical expressions as part of test design
 - You will often find mistakes
- One of the most common is **preconditions** ...

Preconditions

- Programmers often include **preconditions** for their methods
- The preconditions are often expressed in **comments** in method headers
- Preconditions can be in **javadoc**, “requires”, “pre”, ...

Example – Saving addresses

```
// name must not be empty  
// state must be valid  
// zip must be 5 numeric digits  
// street must not be empty  
// city must not be empty
```

Rewriting to logical expression

```
name != "" ∧ state in stateList ∧ zip >= 00000 ∧ zip <= 99999 ∧  
street != "" ∧ city != ""
```

Shortcut for Predicates in Conjunctive Normal Form

- Conjunctive clauses are connected only by the **and** operator
 - $A \wedge B \wedge C \wedge \dots$
- Each major clause is made active by making all other clauses **true**
- ACC tests are “**all true**” and then a “**diagonal**” of false values:

	A	B	C	...
1	T	T	T	...
2	F	T	T	
3	T	F	T	
4	T	T	F	
		.		.
		.		.
		.		.

Shortcut for Predicates in Disjunctive Normal Form

- Disjunctive clauses are connected only by the **or** operator
 - $A \vee B \vee C \vee \dots$
- Each major clause is made active by making all other clauses **false**
- ACC tests are “all **false**” and then a “**diagonal**” of **true** values:

	A	B	C	...
1	F	F	F	...
2	T	F	F	
3	F	T	F	
4	F	F	T	
		.		.
		.		.
		.		.

Summary : Logic Coverage for Specs

- Logical specifications can come from **lots of places** :
 - Preconditions
 - Java asserts
 - Contracts (in design-by-contract development)
 - OCL conditions
 - Formal languages
- Logical specifications can describe behavior at **many levels** :
 - Methods and classes (unit and module testing)
 - Connections among classes and components
 - System-level behavior
- Many predicates in specifications are in **disjunctive** normal or **conjunctive** normal form – simplifying the computations

Introduction to Software Testing

Chapter 3.5

Logic Coverage for FSMs

Paul Ammann & Jeff Offutt

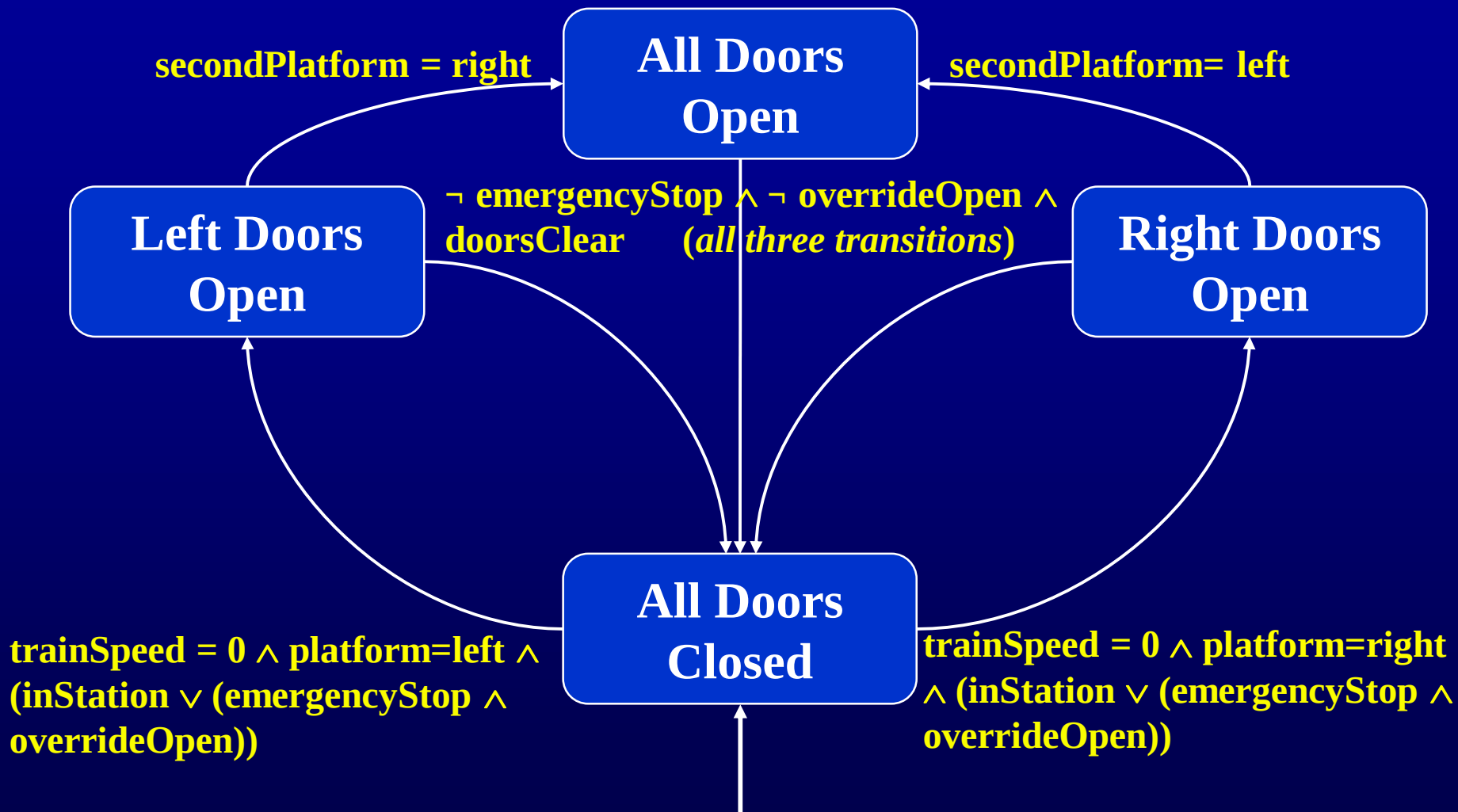
<http://www.cs.gmu.edu/~offutt/softwaretest/>

Covering Finite State Machines

- FSMs are graphs
 - nodes represent state
 - edges represent transitions among states
- Transitions often have logical expressions as guards or triggers
- As we said:

Find a *logical expression* and cover it

Example – Subway Train(黃禹程講)



Determination of the Predicate (郭世揚)

trainSpeed = 0 $\wedge$ **platform=left** $\wedge$ (**inStation** $\vee$ (**emergencyStop** $\wedge$ **overrideOpen**))

trainSpeed = 0 : **platform = left** $\wedge$ (**inStation** $\vee$ (**emergencyStop** $\wedge$ **overrideOpen**))

platform = left : **trainSpeed = 0** $\wedge$ (**inStation** $\vee$ (**emergencyStop** $\wedge$ **overrideOpen**))

inStation : **trainSpeed = 0** $\wedge$ **platform = left** $\wedge$ ($\neg$ **emergencyStop** $\vee$ $\neg$ **overrideOpen**)

emergencyStop : **trainSpeed = 0** $\wedge$ **platform = left** $\wedge$ ($\neg$ **inStation** $\wedge$ **overrideOpen**)

overrideOpen : **trainSpeed = 0** $\wedge$ **platform = left** $\wedge$ ($\neg$ **inStation** $\wedge$ **emergencyStop**)

Test Truth Assignments (CACCC)(鄭惟浩)

$\text{trainSpeed} = 0 \wedge \text{platform} = \text{left} \wedge (\text{inStation} \vee (\text{emergencyStop} \wedge \text{overrideOpen}))$

	trainSpeed=0	platform=left	inStation	emergencyStop	overrideOpen
trainSpeed = 0	T	t	t	t	t
trainSpeed != 0	F	t	t	t	t
platform = left	duplicate	T	t	t	t
platform != left	t	F	t	t	t
inStation	t	t	T	f	f
$\neg$ inStation	t	t	F	f	f
emergencyStop	t	t	f	T	t
$\neg$ emergencyStop	t	t	f	F	t
overrideOpen	duplicate	t	f	t	T
$\neg$ overrideOpen	t	t	f	t	F

Note : other choices are possible

Complicating Issues

- Some buttons must be pressed simultaneously to have effect – so timing must be tested
- Reachability : The tests must reach the state where the transition starts (the prefix)
- Exit : Some tests must continue executing to an end state
- Expected output : The expected output is the state that the transition reaches for true values, or same state for false values
- Accidental transitions : Sometimes a false value for one transition happens to be a true value for another
 - The alternate expected output must be recognized

Reachability via weakest precondition (1/2)

- State (a) $\rightarrow$ (b): precondition: $x < 5$; action: $x := 0$;
- State (b) $\rightarrow$ (c): precondition: $x > y$; action: $x := 3$;
- Assume that at (c), we want to test $x > z$.
- Assume initial state is (a).

Weakest precondition:

1. The precondition of $x := 3$ to $x > z$: $x > z[x=3] = 3 > z$
2. The precondition of $(x > y)$ to $3 > z$: $x > y \ \&\& \ 3 > z$
3. The precondition of $x := 0$ to $x > y \ \&\& \ 3 > z$: $0 > y \ \&\& \ 3 > z$
4. The precondition: $(x < 5)$ to $0 > y \ \&\& \ 3 > z$: $x < 5 \ \&\& \ 0 > y \ \&\& \ 3 > z$
5. So we need input of x, y, z to make $x < 5 \ \&\& \ 0 > y \ \&\& \ 3 > z$ true at initial state (a).
6. $x=0, y = -1, z = 0$ as test input to check $x > z$ at (c).

Reachability via weakest precondition (2/2)

- Assume that at (c), we want to test $\cos(x)+a*b > z$.
- Assume initial state is (a).

Weakest precondition:

1. The precondition of $x:=3$ to $\cos(x)+a*b>z$: $\text{>z}[x=3] = \cos(3)+a*b>z$
2. The precondition of $(x>y)$ to $3>z$: $x>y \&\& \cos(3)+a*b>z$
3. The precondition of $x:=0$ to $x>y \&\& 3>z$: $0>y \&\& \cos(3)+a*b>z$
4. The precondition: $(x<5)$ to $0>y \&\& 3>z$:
 $x<5 \&\& 0>y \&\& \cos(3)+a*b>z$
5. So we need input of x, y, z to make $x<5 \&\& 0>y \&\& \cos(3)+a*b>z$ true at initial state (a).
6. $x=0, y = -1, z = 0, a=b=2$ as test input to check $x>z$ at (c).

Test Scripts

- **Test scripts** are executable sequences of value assignments
- **Mapping problem** : The names used in the FSMs may not match the names in the program
 - Sometimes a **direct name-to-name mapping** can be found
 - Sometimes **more complicated** actions must be taken to assign the appropriate values
 - **Simulation** : Directly inserting value assignments into the middle of the program
- The solution to this is **implementation-specific**

Summary FSM Logic Testing

- FSMs are **widely used** at all levels of abstraction
- Many ways to **express** FSMs
 - Statecharts, tables, Z, decision tables, Petri nets, ...
- Predicates are usually **explicitly included** on the transitions
 - **Guards**
 - **Actions**
 - Often represent **safety constraints**
- FSMs are often used in **embedded** software

Introduction to Software Testing

Chapter 3.6

Disjunctive Normal Form Criteria

Paul Ammann & Jeff Offutt

<http://www.cs.gmu.edu/~offutt/softwaretest/>

2013/12/19 stopped here.

Disjunctive Normal Form (DNF)

- Common Representation for Boolean Functions
 - Slightly Different Notation for Operators
 - Slightly Different Terminology
- Basics:
 - A **literal** is a clause or the negation (overstrike) of a clause
 - Examples: $a, \bar{a}$
 - A **term** is a set of literals connected by logical “and”
 - “and” is denoted by adjacency instead of $\wedge$
 - Examples: $ab, a\bar{b}, \bar{a}\bar{b}$ for $a \wedge b, a \wedge \neg b, \neg a \wedge \neg b$
 - A **(disjunctive normal form) predicate** is a set of terms connected by “or”
 - “or” is denoted by $+$ instead of $\vee$
 - Examples: $abc + \bar{a}b + a\bar{c}$
 - Terms are also called “implicants”
 - If a term is true, that implies the predicate is true

Disjunctive Normal Form (DNF)

Examples:

- $abc + \sim ab + b\sim c$
- $ab + \sim b\sim c$
- $a(b + \sim c)$ not DNF $\rightarrow$ distributivity: $ab + a\sim c$: DNF
- $\sim(a + \sim c)$ not DNF $\rightarrow$ de Morgan's law: $\sim a \sim \sim c \rightarrow \sim ac$: DNF

Properties of DNF:

- $a + ab \equiv a$
- $a + a\sim b \equiv a$
- $a\sim b + b \equiv a + b$

Implicant Coverage (張唯霖講)

- Obvious coverage idea: Make each implicant evaluate to “true”.
 - Problem: Only tests “true” cases for the predicate.
 - Solution: Include DNF representations for negation.

Implicant Coverage (IC) : Given DNF representations of a predicate f and its negation $\bar{f}$, for each implicant in f and $\bar{f}$, TR contains the requirement that the implicant evaluate to true.

- Example: $f = ab + b\bar{c}$ $\bar{f} = \bar{b} + \bar{a}c$
 - Implicants: $\{ ab, b\bar{c}, \bar{b}, \bar{a}c \}$
 - Possible test set: $\{TTF, FFT\}$
- Observation: IC is relatively weak

Improving on Implicant Coverage (Martin講)

- Additional Definitions:

- A **proper subterm** is a term with one or more clauses removed
 - Example: abc has 6 proper subterms: a, b, c, ab, ac, bc
- A **prime implicant** is an implicant such that no proper subterm is also an implicant (in any DNF of the predicate).
 - Example: $f = ab + \overline{a}bc$
 - Implicant ab is a prime implicant
 - Implicant $\overline{a}bc$ is not a prime implicant (due to proper subterm ac)
 - $f = ab + \overline{a}bc = a(b + \overline{b}c) = a(b + c) = ab + ac$
- A **redundant implicant** is an implicant that can be removed without changing the value of the predicate
 - Example: $f = ab + ac + \overline{b}c$
 - ab is redundant
 - Predicate can be written: $ac + \overline{b}c$

**ab is redundant since $1c + 1\sim c$ is true.
Finding minimum DNF is NP-hard.**

Unique True Points (吳一鵬講)

- A **minimal DNF representation** is one with only prime, nonredundant implicants.
- A **unique true point** with respect to a given implicant is an assignment of truth values so that
 - the given implicant is true, and
 - all other implicants are false
- Hence a unique true point test focuses on just one implicant
- A minimal representation guarantees the existence of at least one unique true point for each implicant

Unique True Point Coverage (UTPC) : Given minimal DNF representations of a predicate f and its negation $\bar{f}$, TR contains a unique true point for each implicant in f and $\bar{f}$.

Unique True Point Example (林柏僑講)

- Consider again: $f = ab + b\bar{c}$ $\bar{f} = \bar{b} + \bar{a}c$
 - Implicants: $\{ab, b\bar{c}, \bar{b}, \bar{a}c\}$
 - Each of these implicants is prime
 - None of these implicants is redundant
- Unique true points:
 - ab : {TTT}
 - $b\bar{c}$: {FTF}
 - $\bar{b}$: {FFF, TFF, TFT}
 - $\bar{a}c$: {FTT}
- Note that there are three possible (minimal) tests satisfying UTPC
- UTPC is fairly powerful
 - Exponential in general, but reasonable cost for many common functions
 - No subsumption relation wrt any of the ACC or ICC Criteria

Near False Points

- A **near false point** with respect to a clause c in implicant i is an assignment of truth values such that f is false, but if c is negated (and all other clauses left as is), i (and hence f) evaluates to true.
- Relation to **determination**: at a near false point, c determines f
 - Hence we should expect relationship to ACC criteria

Unique True Point and Near False Point Pair Coverage (CUTPNFP) : Given a minimal DNF representation of a predicate f , for each clause c in each implicant i , TR contains a unique true point for i and a near false point for c such that the points differ only in the truth value of c .

- Note that definition only mentions f , and not $\bar{f}$.
- Clearly, CUTPNFP subsumes RACC

CUTPNFP Example

- Consider $f = ab + cd$
 - For implicant ab , we have 3 unique true points: {TTFF, TTFT, TTTF}
 - For clause a , we can pair unique true point TTFF with near false point FTF
 - For clause b , we can pair unique true point TTFF with near false point TTFF
 - For implicant cd , we have 3 unique true points: {FFTT, FTTF, TFFT}
 - For clause c , we can pair unique true point FFTT with near false point FFT
 - For clause d , we can pair unique true point FFTT with near false point FFTF
- CUTPNFP set: {TTFF, FFTT, TFFF, FTFF, FFTF, FFFT}
 - First two tests are unique true points; others are near false points
- Rough number of tests required: # implicants * # literals

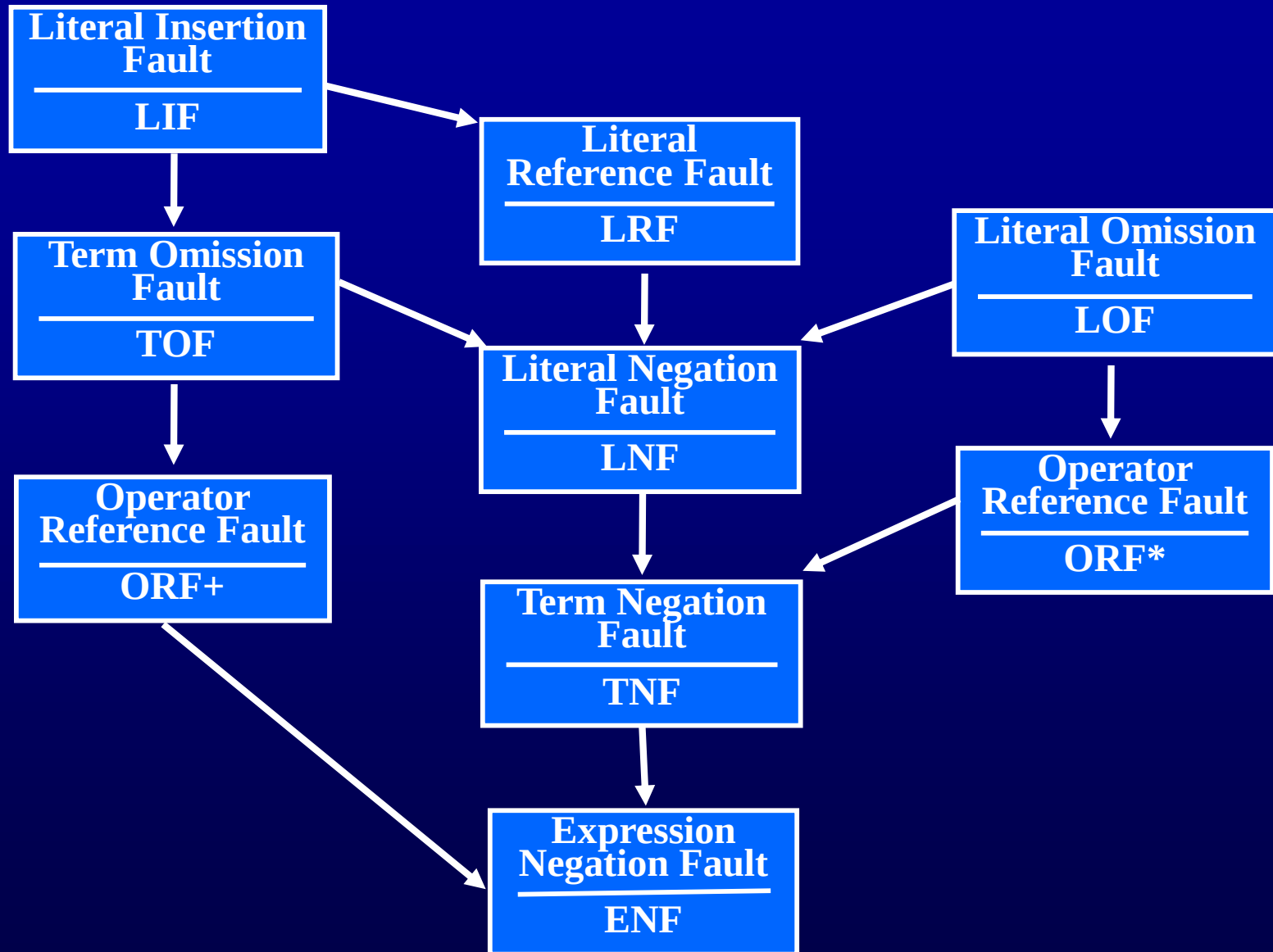
DNF Fault Classes

• ENF: Expression Negation Fault	$f = ab+c$	$f' = \overline{ab+c}$
• TNF: Term Negation Fault	$f = ab+c$	$f' = \overline{ab}+c$
• TOF: Term Omission Fault	$f = ab+c$	$f' = ab$
• LNF: Literal Negation Fault	$f = ab+c$	$f' = a\bar{b}+c$
• LRF: Literal Reference Fault	$f = ab + bcd$	$f' = ad + bcd$
• LOF: Literal Omission Fault	$f = ab + c$	$f' = a + c$
• LIF: Literal Insertion Fault	$f = ab + c$	$f' = ab + bc$
• ORF+: Operator Reference Fault	$f = ab + c$	$f' = abc$
• ORF*: Operator Reference Fault	$f = ab + c$	$f' = a + b + c$

Key idea is that fault classes are related with respect to testing:

Test sets guaranteed to detect certain faults are also guaranteed to detect additional faults.

Fault Detection Relationships



Understanding The Detection Relationships

- Consider the TOF (Term Omission Fault) class
 - UTPC requires a unique true point for every implicant (term)
 - Hence UTPC detects all TOF faults
 - From the diagram, UTPC also detects:
 - All LNF faults (Unique true point for implicant now false)
 - All TNF faults (All true points for implicant are now false points)
 - All ORF+ faults (Unique true points for joined terms now false)
 - All ENF faults (Any single test detects this...)
- Although CUTPNFP does not subsume UTPC, CUTPNFP detects all fault classes that UTPC detects (Converse is false)
- Consider what this says about the notions of subsumption vs. fault detection
- Literature has many more powerful (and more expensive) DNF criteria
 - In particular, possible to detect entire fault hierarchy (MUMCUT)

Karnaugh Maps for Testing Logic Expressions

- Fair Warning
 - We *use*, rather than *present*, Karnaugh Maps
 - Newcomer to Karnaugh Maps probably needs a tutorial
 - Suggestion: Google “Karnaugh Map Tutorial”
- Our goal: Apply Karnaugh Maps to concepts used to test logic expressions
 - Identify when a clause determines a predicate
 - Identify the negation of a predicate
 - Identify prime implicants and redundant implicants
 - Identify unique true points
 - Identify unique true point / near false point pairs
- No new material here on *testing*
 - Just fast shortcuts for concepts already presented

K-Map: A clause determines a predicate

- Consider the predicate: $f = b + \bar{a}\bar{c} + ac$
- Suppose we want to identify when b determines f
- The dashed line highlights where b changes value
 - If two cells joined by the dashed line have different values for f , then b determines f for those two cells.
 - b determines f : $\bar{a}\bar{c} + ac$ (but NOT at ac or $\bar{a}\bar{c}$)
- Repeat for clauses a and c

ab					
		00	01	11	10
c	0	t	t	t	t
	1	t	t	t	t

K-Map: Negation of a predicate

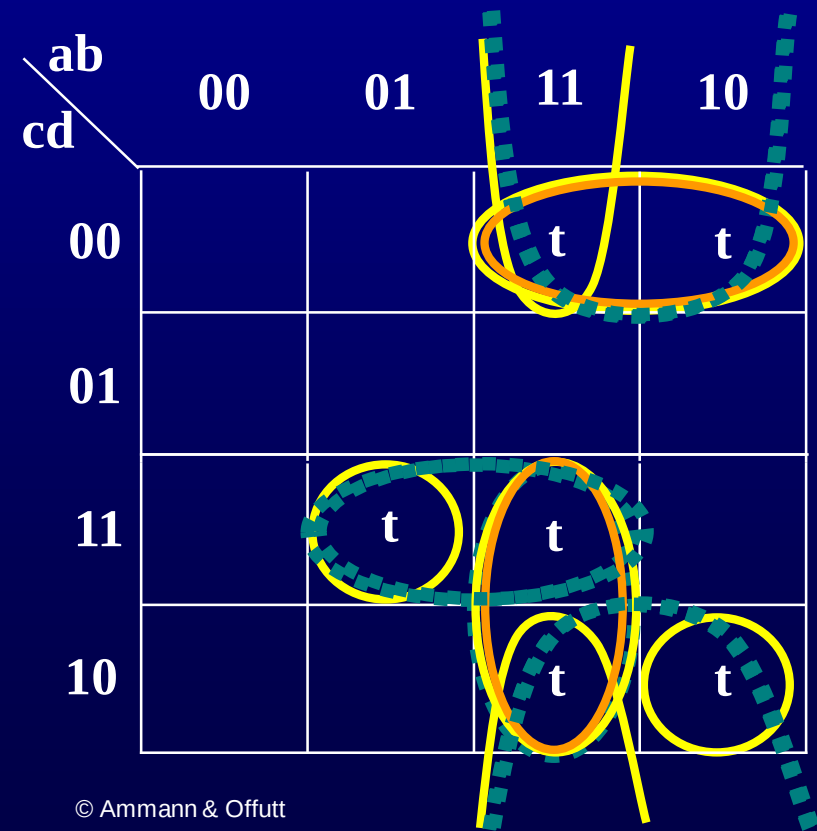
- Consider the predicate: $f = ab + bc$
- Draw the Karnaugh Map for the negation
 - Identify groups
 - Write down negation: $\bar{f} = \bar{b} + \bar{a}\bar{c}$

ab \ c	00	01	11	10
0			t	
1		t	t	

ab \ c	00	01	11	10
0	t	t		t
1	t			t

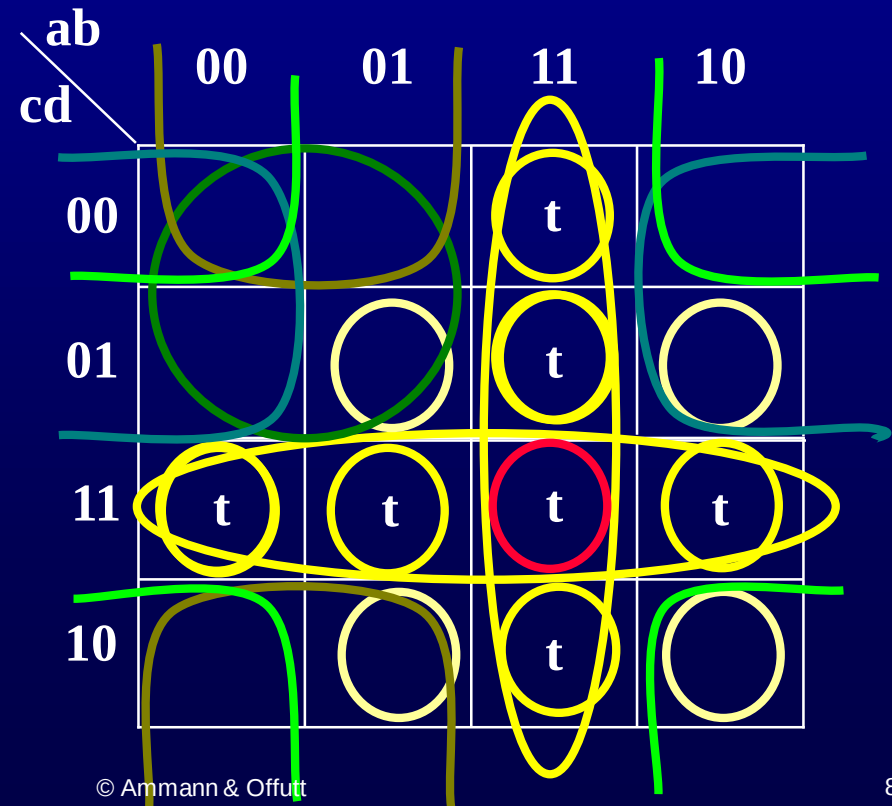
K-Map: Prime and redundant implicants

- Consider the predicate: $f = \underline{abc} + \underline{abd} + \underline{\bar{a}bcd} + \underline{\bar{a}cd} + \underline{acd}$
- Draw the Karnaugh Map
- Implicants that are not prime: $\underline{abd}$, $\underline{\bar{a}bcd}$, $\underline{\bar{a}cd}$, $\underline{acd}$
- Redundant implicant: $\underline{abd}$
- Prime implicants
 - Three: $\bar{a}d$, bcd , abc
 - The last is redundant
 - Minimal DNF representation
 - $f = \bar{a}d + bcd$



K-Map: Unique True Points

- Consider the predicate: $f = ab + cd$
- Three unique true points for ab
 - TTFF, TTFT, TTTF
 - TTTT is a true point, but not a unique true point
- Three unique true points for cd
 - FFTT, FTTT, TFTT
- Unique true points for $\bar{f}$
 $\bar{f} = \bar{a}\bar{c} + \bar{b}\bar{c} + \bar{a}\bar{d} + \bar{b}\bar{d}$
 - FTFT, TFFT, FTTF, TFTF
- Possible UTPC test set
 - f : {TTFT, FFTT}
 - $\bar{f}$: {FTFT, TFFT, FTTF, TFTF}



K-Map: Unique True Point/ Near False Point Pairs

- Consider the predicate: $f = ab + cd$
- For implicant ab
 - For a , choose UTP, NFP pair
 - TTFF, FTFF
 - For b , choose UTP, NFP pair
 - TTFT, TFFT
- For implicant cd
 - For c , choose UTP, NFP pair
 - FFTT, FFFT
 - For d , choose UTP, NFP pair
 - FFTT, FFTE
- Possible CUTPNFP test set
 - {TTFF, TTFT, FFTT} //UTPs
 - {FTFF, TFFT, FFFT, FFTE} //NFPs

ab cd	00	01	11	10
00			t	
01			t	
11	t	t	t	t
10			t	